



الكلية التطبيقية
الجامعة السعودية الإلكترونية
SAUDI ELECTRONIC UNIVERSITY
2011-1432

دبلوم الذكاء الاصطناعي التوليدي

اسم المقرر: البرمجة باستخدام بايثون

رمز المقرر: PYT103

الموضوع: هياكل البيانات – القواميس والمجموعات

الموضوع الرابع
عشر: هياكل البيانات
– القواميس
والمجموعات



الأهداف

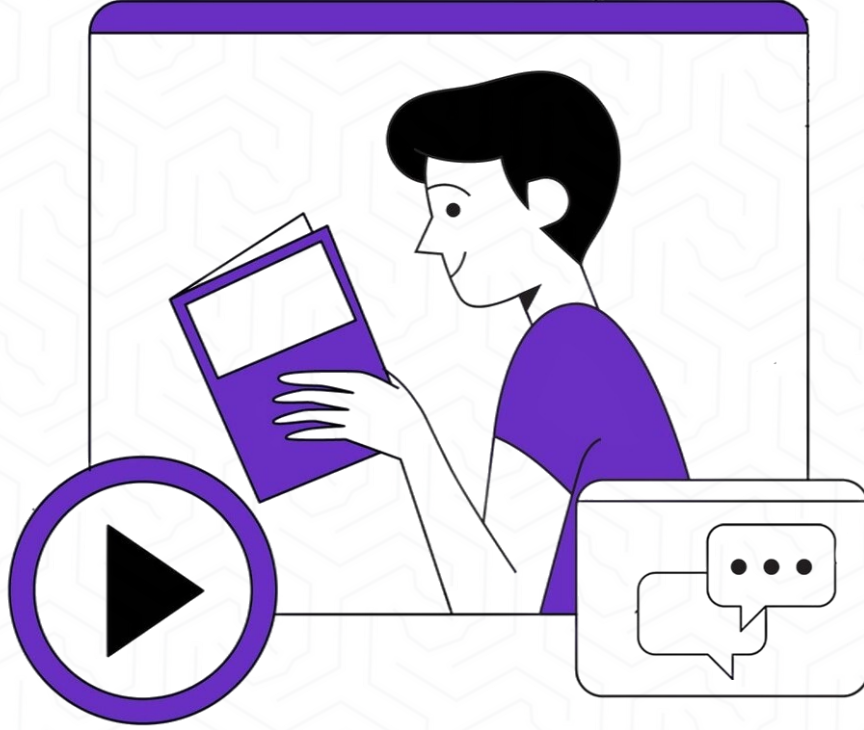
في نهاية هذا الدرس سيكون الطالب قادرًا على أن:

- ✓ فهم مفهوم القواميس Dictionaries وبنيتها الأساسية في Python.
- ✓ إنشاء وتعديل القواميس باستخدام طرق مختلفة.
- ✓ الوصول إلى عناصر القاموس وتحديثها وحذفها.
- ✓ استخدام دوال ووظائف القواميس المدمجة.
- ✓ فهم مفهوم المجموعات Sets وخصائصها الفريدة.
- ✓ إنشاء المجموعات وإجراء العمليات الرياضية عليها.
- ✓ التمييز بين القواميس والمجموعات والقوائم.



وصف الدرس

في هذا الدرس سوف نتناول :



تعتبر القواميس Dictionaries والمجموعات Sets من أقوى هياكل البيانات في لغة Python، حيث توفر طرقاً فعّالة ومرنة لتنظيم وإدارة البيانات. القاموس هو هيكل بيانات يخزن البيانات على شكل أزواج مفتاح-قيمة (key-value pairs)، مما يتيح الوصول السريع للبيانات باستخدام مفاتيح فريدة بدلاً من الفهارس العددية كما في القوائم. هذه الخاصية تجعل القواميس مثالية لتمثيل بيانات منظمة مثل معلومات المستخدمين، إعدادات التطبيقات، وقواعد البيانات البسيطة. أما المجموعات فهي هياكل بيانات غير مرتبة تحتوي على عناصر فريدة فقط، وتستخدم بشكل أساسي لإزالة التكرارات وإجراء العمليات الرياضية مثل الاتحاد، التقاطع، والفرق.



مقدمة إلى القواميس Dictionaries

المبدأ والفكرة:

القاموس في Python هو هيكل بيانات يخزن البيانات كأزواج مفتاح-قيمة key-value pair كل مفتاح فريد ويستخدم للوصول إلى قيمته المرتبطة به.

الخصائص الأساسية:

غير مرتب: العناصر ليس لها ترتيب ثابت قبل Python 3.7، مرتبة بالإدخال من Python 3.7+.
قابل للتعديل: يمكن إضافة وحذف وتعديل العناصر.
مفاتيح فريدة: كل مفتاح يجب أن يكون فريداً.
مفاتيح غير قابلة للتغيير: المفاتيح يجب أن تكون من أنواع (strings, numbers, tuples) immutable.

```
dictionary = {key1: value1, key2: value2, key3: value3}
```


أمثلة تطبيقية

قاموس أسعار المنتجات

```
prices = {  
    "apple": 5.5,  
    "banana": 3.0,  
    "orange": 4.5,  
    "mango": 8.0  
}
```

```
print(prices)
```

النتيجة: {'apple': 5.5, 'banana': 3.0, 'orange': 4.5, 'mango': 8.0}

مثال: قاموس فارغ

```
empty_dict = {}
```

print(type(empty_dict)) # النتيجة: <class 'dict'>

مثال: قاموس بأنواع مختلفة من القيم

```
person = {  
    "name": "سارة",  
    "age": 25,  
    "is_student": True,  
    "grades": [85, 90, 88],  
    "address": {  
        "city": "الرياض",  
        "street": "الملك فهد"  
    }  
}
```

```
print(person)
```

القاموس يمكن أن يحتوي على أي نوع بيانات كقيم

مثال: إنشاء قاموس من قوائم
keys = ["name", "age", "city"]
values = ["جدة", "30", "محمد"]
person_dict = dict(zip(keys, values))

```
print(person_dict)
```

النتيجة: {'name': 'محمد', 'age': 30, 'city': 'جدة'}

الوصول إلى عناصر القاموس

المبدأ والفكرة:

للوصول إلى قيمة معينة في القاموس، نستخدم المفتاح المرتبط بها.

هناك طريقتان رئيسيتان:

1. استخدام الأقواس المربعة []

2. استخدام دالة get()

الفرق:

• [] تُرجع خطأ KeyError إذا لم يوجد

المفتاح

• get() تُرجع None أو قيمة افتراضية

إذا لم يوجد المفتاح

```
# إنشاء قاموس
student = {
    "name": "فاطمة",
    "age": 22,
    "major": "Engineering",
    "gpa": 3.9,
    "graduated": False
}

# مثال 1: الوصول باستخدام []
print(student["name"]) # النتيجة: فاطمة
print(student["age"]) # النتيجة: 22
print(student["gpa"]) # النتيجة: 3.9

# مثال 2: محاولة الوصول لمفتاح غير موجود
try:
    print(student["phone"]) # سيجري خطأ
except KeyError:
    print("المفتاح غير موجود")

# مثال 3: الوصول باستخدام get()
print(student.get("name")) # النتيجة: فاطمة
print(student.get("phone")) # النتيجة: None (بدون خطأ)

# مثال 4: مع قيمة افتراضية
phone = student.get("phone", "غير متوفر")
print(phone) # النتيجة: غير متوفر
```


تعديل وإضافة عناصر القاموس

المبدأ والفكرة:

القواميس قابلة للتعديل, mutable, مما يعني يمكننا:

تعديل قيمة موجودة

إضافة زوج مفتاح-قيمة جديد

تحديث عدة عناصر دفعة واحدة

الطرق:

- dict[key] = value للإضافة أو التعديل

- update() لتحديث عدة عناصر

- setdefault() لإضافة قيمة فقط إذا لم يكن المفتاح موجوداً

مثال 1: تعديل قيمة موجودة #

```
student = {  
    "name": "خالد",  
    "age": 21,  
    "gpa": 3.5  
}
```

```
print("قبل التعديل:", student)  
student["gpa"] = 3.8 # تعديل المعدل  
print("بعد التعديل:", student)
```

مثال 2: إضافة عنصر جديد #

```
student["major"] = "Mathematics"  
student["phone"] = "0501234567"
```

```
print("بعد الإضافة:", student)
```

```
# النتيجة: {'name': 'خالد', 'age': 21, 'gpa':  
3.8, 'major': 'Mathematics', 'phone':  
'0501234567'}
```

لتحديث عدة عناصر update() مثال 3: استخدام #

```
student.update({  
    "age": 22,  
    "city": "الدمام",  
    "email": "khaled@example.com"  
})
```

```
print("بعد update:", student)
```

مع قاموس update() مثال 4: استخدام #
آخر

```
additional_info = {  
    "graduation_year": 2025,  
    "honors": True  
}
```

```
student.update(additional_info)  
print(student)
```

إضافة فقط إذا - setdefault() مثال 5 #
لم يكن موجوداً

```
student.setdefault("nationality",  
    "سعودي") # سيُضاف  
student.setdefault("name", "علي")  
# موجود name لن يتغير لأن
```

```
print("الجنسية:",  
student["nationality"]) # النتيجة :  
سعودي
```

```
print("الاسم:", student["name"])  
# النتيجة: خالد (لم يتغير)
```

حذف عناصر من القاموس

المبدأ والفكرة:

هناك عدة طرق لحذف عناصر من القاموس:

- del - حذف عنصر بمفتاح محدد
- pop() - حذف وإرجاع القيمة
- popitem() - حذف وإرجاع آخر زوج مفتاح-قيمة
- clear() - حذف جميع العناصر

```
# مع قيمة افتراضية pop()
english_score = scores.pop("english", 0)
print(f"النتيجة: 0: {english_score}") # درجة الإنجليزي
```

```
# popitem() مثال 4: استخدام
courses = {
    "CS101": "Introduction to Programming",
    "CS102": "Data Structures",
    "CS103": "Algorithms"
}
```

```
last_item = courses.popitem() # يحذف آخر عنصر (في Python 3.7+)
print(f"العنصر المحذوف: {last_item}")
print("بعد popitem:", courses)
```

```
# clear() مثال 5: استخدام
inventory = {
    "apples": 50,
    "bananas": 30,
    "oranges": 25
}

print("قبل clear:", inventory)
inventory.clear() # حذف جميع العناصر
print("النتيجة: {clear: بعد", inventory)
```

```
# del مثال 1: استخدام
student = {
    "name": "نورة",
    "age": 23,
    "major": "Physics",
    "temp_id": "12345"
}
```

```
print("قبل الحذف:", student)
del student["temp_id"] # حذف المعرف المؤقت
print("بعد del:", student)
```

```
# مثال 2: حذف القاموس بالكامل
temp_dict = {"a": 1, "b": 2}
del temp_dict
# سيجر خطأ لأن القاموس # print(temp_dict)
حذف
```

```
# pop() مثال 3: استخدام
scores = {
    "math": 95,
    "physics": 88,
    "chemistry": 92,
    "biology": 85
}
```

```
# يحذف ويُرجع القيمة pop()
chemistry_score = scores.pop("chemistry")
print(f"درجة الكيمياء: {chemistry_score}") #
النتيجة: 92
print("بعد pop:", scores)
```


دوال القواميس المدمجة

المبدأ والفكرة:

Python توفر دوال مدمجة قوية للعمل مع

القواميس:

- `keys()` إرجاع جميع المفاتيح

- `values()` إرجاع جميع القيم

- `items()` إرجاع أزواج المفتاح-القيمة

- `len()` عدد العناصر

```
# إنشاء قاموس للمثال
grades = {
    "85": أحمد,
    "92": فاطمة,
    "78": خالد,
    "95": سارة,
    "88": محمد
}

# الحصول على جميع المفاتيح - keys(): مثال 1
print("المفاتيح:", grades.keys())
# dict_keys(['محمد', 'سارة', 'خالد', 'فاطمة', 'أحمد']) النتيجة:

# تحويل إلى قائمة
students_list = list(grades.keys())
print("قائمة الطلاب:", students_list)

# الحصول على جميع القيم - values(): مثال 2
print("الدرجات:", grades.values())
# dict_values([85, 92, 78, 95, 88]) النتيجة:

# حساب متوسط الدرجات
average = sum(grades.values()) / len(grades)
print(f"المتوسط: {average}") # 87.6 النتيجة:

# الحصول على الأزواج - items(): مثال 3
print("الأزواج:", grades.items())
# dict_items([('85', أحمد), ('92', فاطمة), ('78', خالد), ('95', سارة), ('88', محمد)]) النتيجة:
```

مقدمة إلى المجموعات

المبدأ والفكرة:

المجموعة (Set) هي هيكل بيانات غير مرتب يحتوي على عناصر فريدة فقط. تُستخدم المجموعات لإزالة التكرار وإجراء عمليات رياضية.

الخصائص:

غير مرتبة: لا يوجد فهرس للعناصر

عناصر فريدة: لا يسمح بالتكرار

قابلة للتعديل: يمكن إضافة وحذف العناصر

عناصر غير قابلة للتغيير: العناصر يجب أن تكون

immutable

```
my_set = {element1, element2, element3}
```


مثال تطبيقي

مثال 1: إنشاء مجموعة بسيطة
fruits = {"apple", "banana", "cherry"}
print("المجموعة:", fruits)
print("النوع:", type(fruits))

مثال 2: المجموعة تزيل التكرارات تلقائياً
numbers = {1, 2, 3, 2, 4, 1, 5, 3}
print("الأرقام (بدون تكرار):", numbers)
النتيجة: {5, 4, 3, 2, 1}

مثال 3: إنشاء مجموعة فارغة
لاحظ: {} تَنشئ قاموس فارغ، ليس مجموعة!
empty_set = set() # الطريقة الصحيحة
print("المجموعة فارغة:", empty_set)
print("النوع:", type(empty_set))

مثال 4: تحويل قائمة إلى مجموعة (لإزالة التكرار)
colors_list = ["red", "blue", "red", "green", "blue", "yellow"]
colors_set = set(colors_list)
print("القائمة الأصلية:", colors_list)
print("المجموعة (بدون تكرار):", colors_set)

تحويل مرة أخرى لقائمة
unique_colors = list(colors_set)
print("قائمة فريدة:", unique_colors)

مثال 5: مجموعة من نص (characters)
text = "hello"
char_set = set(text)
print("أحرف النص:", char_set)
ملاحظة: ' ' فقط 'الاحظ' - ' ' 'o', 'l', 'e', 'h' النتيجة: ' ' }

مثال 6: مجموعة من أنواع بيانات مختلفة
mixed_set = {1, "hello", 3.14, True, (1, 2)}
print("مجموعة مختلطة:", mixed_set)

ملاحظة: لا يمكن إضافة قوائم للمجموعة (لأنها mutable)
TypeError سيُرجع خطأ # error_set = {1, 2, [3, 4]}



إضافة وحذف عناصر المجموعات

المبدأ والفكرة:

المجموعات قابلة للتعديل، يمكننا إضافة وحذف العناصر باستخدام عدة دوال:

الإضافة:

- add() إضافة عنصر واحد
 - update() إضافة عدة عناصر
- ## الحذف:

- remove() حذف عنصر (خطأ إذا لم يوجد)
- discard() حذف عنصر (بدون خطأ)
- pop() حذف عنصر عشوائي
- clear() حذف جميع العناصر

```
# إضافة عناصر باستخدام add()
fruits = {"apple", "banana"}
print("قبل الإضافة:", fruits)
```

```
fruits.add("orange")
fruits.add("mango")
print("بعد الإضافة:", fruits)
```

```
# إضافة عنصر موجود (لن يتكرر)
fruits.add("apple")
print("لم يتغير # مجدداً apple بعد إضافة", fruits)
```

```
# مثال 2: إضافة عدة عناصر باستخدام update()
fruits.update(["grape", "kiwi", "pear"])
print("بقائمة update بعد:", fruits)
```

```
# iterable من أي update يمكن
fruits.update({"melon", "berry"}) # مجموعة
fruits.update("xy") # نص 'x' و 'y' (يضيف)
print("بعد عدة updates:", fruits)
```

```
# مثال 3: حذف عنصر باستخدام remove()
numbers = {1, 2, 3, 4, 5}
print("قبل الحذف:", numbers)
```

```
numbers.remove(3)
print("بعد remove(3):", numbers)
```

```
# تَرجع خطأ إذا لم يوجد العنصر remove()
try:
    numbers.remove(10)
except KeyError:
    print("العنصر 10 غير موجود")
```

```
# أكثر أماناً discard() مثال 4: حذف عنصر باستخدام
numbers.discard(5)
print("بعد discard(5):", numbers)
```

```
لا يُرجع خطأ حتى لو لم يوجد
numbers.discard(100) #
print("بعد discard(100):", numbers) # لم يتغير
```

```
# مثال 5: حذف عنصر عشوائي باستخدام pop()
colors = {"red", "green", "blue", "yellow"}
print("المجموعة:", colors)
```

```
removed_color = colors.pop()
print(f"العنصر المحذوف: {removed_color}")
print("بعد pop:", colors)
```

```
# مثال 6: حذف جميع العناصر باستخدام clear()
temp_set = {1, 2, 3, 4, 5}
print("قبل clear:", temp_set)
temp_set.clear()
print("بعد clear:", temp_set) # النتيجة: set()
```


عمليات المجموعات الرياضية

المبدأ والفكرة:

المجموعات تدعم العمليات الرياضية الأساسية:

• الاتحاد - `union()` جميع العناصر من المجموعتين

• التقاطع - `intersection()` العناصر المشتركة فقط

• الفرق - `difference()` العناصر في الأولى وليست في الثانية

• الفرق المتماثل Symmetric Difference): `symmetric_difference()` أو العناصر في إحدهما فقط

إنشاء مجموعات للأمثلة #

```
set_a = {1, 2, 3, 4, 5}
```

```
set_b = {4, 5, 6, 7, 8}
```

```
print("المجموعة A:", set_a)
```

```
print("المجموعة B:", set_b)
```

جميع العناصر - (Union) مثال 1: الاتحاد #

```
union1 = set_a | set_b
```

```
union2 = set_a.union(set_b)
```

```
print("\nالاتحاد A | B:", union1)
```

```
print("الاتحاد A.union(B):", union2)
```

```
# النتيجة: {8, 7, 6, 5, 4, 3, 2, 1}
```

العناصر - (Intersection) مثال 2: التقاطع المشتركة

```
intersection1 = set_a & set_b
```

```
intersection2 = set_a.intersection(set_b)
```

```
print("\nالتقاطع A & B:", intersection1)
```

```
print("التقاطع A.intersection(B):",
```

```
intersection2)
```

```
# النتيجة: {5, 4}
```

وليس A في - (Difference) مثال 3: الفرق في B #

```
difference1 = set_a - set_b
```

```
difference2 = set_a.difference(set_b)
```

```
print("\nالفرق A - B:", difference1)
```

```
print("A.difference(B):",
```

```
difference2)
```

```
# النتيجة: {3, 2, 1}
```

الفرق في الاتجاه الآخر #

```
print("B - A: الفرق", set_b - set_a)
```

```
# النتيجة: {8, 7, 6}
```

(Symmetric Difference) مثال 4: الفرق المتماثل #

العناصر في إحدهما فقط (ليست مشتركة) #

```
sym_diff1 = set_a ^ set_b
```

```
sym_diff2 =
```

```
set_a.symmetric_difference(set_b)
```

```
print("\nالفرق المتماثل A ^ B:",
```

```
sym_diff1)
```

```
print("الفرق المتماثل
```

```
A.symmetric_difference(B):", sym_diff2)
```

```
# النتيجة: {8, 7, 6, 3, 2, 1}
```

المقارنة بين القوائم والقواميس والمجموعات

المبدأ والفكرة:

كل هيكل بيانات له استخداماته المناسبة. فهم الفروقات يساعد في اختيار الأنسب.
جدول المقارنة:

Set	Dictionary	List	الخاصية
X	✓ (Python 3.7+)	✓	مرتب
X	X (بالمفتاح)	✓	قابل للفهرسة
✓	✓	✓	قابل للتعديل
X	X (المفاتيح)	✓	يسمح بالتكرار



Python for Everybody: Exploring Data in Book by Python 3 Charles Severance.

لکم شکراً





الكلية التطبيقية
الجامعة السعودية الإلكترونية
SAUDI ELECTRONIC UNIVERSITY
2011-1432

دبلوم الذكاء الاصطناعي التوليدي

اسم المقرر: البرمجة باستخدام بايثون

رمز المقرر: PYT103

الموضوع: الثاني عشر

الموضوع الثاني
عشر:
إعادة – NumPy
التشكيل، البتّ،
الأقنعة



الأهداف

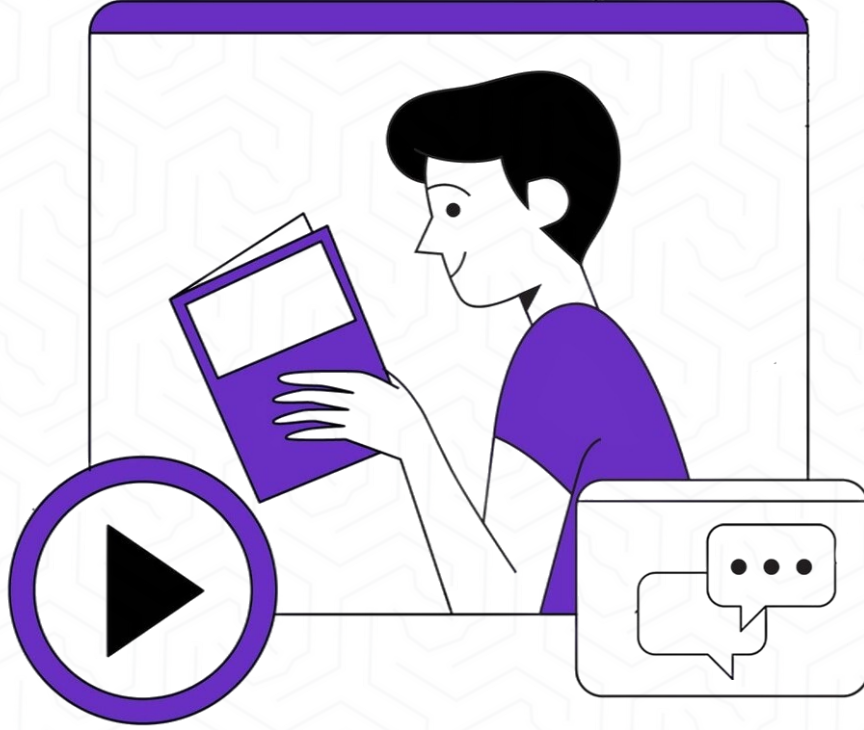
في نهاية هذا الدرس سيكون الطالب قادرًا على أن:

- ✓ يوضح مفهوم إعادة تشكيل المصفوفات Reshaping وتطبيقاته العملية.
- ✓ يستخدم دوال إعادة التشكيل المختلفة مثل `reshape()`, `flatten()`, `ravel()`.
- ✓ يفهم آلية البث Broadcasting في NumPy وقواعدها.
- ✓ يطبق العمليات الحسابية على مصفوفات بأشكال مختلفة.
- ✓ يستخدم الأقنعة المنطقية Boolean Masking لفلتر البيانات.
- ✓ يطبق الفهرسة المتقدمة Advanced Indexing والفهرسة الشرطية باستخدام دوال `where()`, `select()`, `choose()` للتحكم المتقدم.
- ✓ يقسم المصفوفات باستخدام `concatenate()`, `split()`, `stack()`.
- ✓ يطبق عمليات النقل والتبديل Transpose and Swapping وبناء تطبيقات عملية تجمع بين هذه التقنيات



وصف الدرس

في هذا الدرس سوف نتناول :



- التقنيات المتقدمة في مكتبة NumPy للتعامل مع البيانات متعددة الأبعاد بكفاءة عالية.
- مفهوم إعادة التشكيل Reshaping للمصفوفات وكيفية تغيير شكلها دون المساس بمحتواها.
- تطبيقات إعادة التشكيل في معالجة الصور، التعلم الآلي، وتحليل البيانات.
- مفهوم البث Broadcasting كآلية ذكية لتنفيذ العمليات الحسابية بين مصفوفات بأحجام مختلفة دون تكرار البيانات.
- الأقنعة المنطقية Boolean Masking كأداة قوية لفلتر البيانات واختيار العناصر بناءً على شروط منطقية محددة.
- أهمية هذه التقنيات في رفع كفاءة التحليل ومعالجة البيانات باستخدام NumPy



مقدمة إلى إعادة التشكيل Reshaping

المبدأ والفكرة:

إعادة التشكيل Reshaping هي عملية تغيير شكل المصفوفة (عدد الصفوف والأعمدة والأبعاد) دون تغيير البيانات نفسها. هذه العملية ضرورية في:

- تحضير البيانات للنماذج (مثل تحويل صورة إلى متجه)

- إعادة تنظيم البيانات للعمليات الحسابية
- التوافق مع متطلبات المكتبات الأخرى

الشروط المهمة:

- عدد العناصر يجب أن يبقى ثابتاً
- الترتيب الافتراضي: Row-major (C-style)

```
new_array = array.reshape(new_shape)
```


الموضوع الحادي عشر

مقدمة إلى إعادة التشكيل

Reshaping

```
import numpy as np
```

```
# إنشاء مصفوفة أحادية البعد
```

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
print("المصفوفة الأصلية:")
print(arr)
print(f"الشكل: {arr.shape}")
```

```
# (صفوف 4 × أعمدة 3) تحويل إلى مصفوفة 2
```

```
arr_2d = arr.reshape(3, 4)
print("\n4, 3 (بعد إعادة التشكيل إلى)")
print(arr_2d)
print(f"الشكل: {arr_2d.shape}")
```

```
# (بشكل مختلف) 4 صفوف × 3 أعمدة (D تحويل إلى مصفوفة 2)
```

```
arr_2d_alt = arr.reshape(4, 3)
print("\n3, 4 (بعد إعادة التشكيل إلى)")
print(arr_2d_alt)
```

```
# (2 × 2 × 3) D تحويل إلى مصفوفة 3
```

```
arr_3d = arr.reshape(2, 2, 3)
print("\n3, 2, 2 (بعد إعادة التشكيل إلى)")
print(arr_3d)
print(f"الشكل: {arr_3d.shape}")
print(f"عدد الأبعاد: {arr_3d.ndim}")
```

```
# استخدام 1- للحساب التلقائي
```

```
# تعني: احسب هذا البعد تلقائياً بناءً على باقي الأبعاد 1-
```

```
arr_auto = arr.reshape(3, -1) # 3 صفوف، الأعمدة تُحسب تلقائياً
print("\n1-, 3 (بإستخدام 1-)")
print(arr_auto)
print(f"الشكل: {arr_auto.shape}") # 4, 3 (النتيجة)
```

```
arr_auto2 = arr.reshape(-1, 2) # عمودان، الصفوف تُحسب تلقائياً
```

```
print("\n2, 1-( 1- بإستخدام)")
print(arr_auto2)
print(f"الشكل: {arr_auto2.shape}") # 2, 6 (النتيجة)
```

```
# محاولة إعادة تشكيل خاطئة - سبب خطأ
```

```
try:
```

```
    wrong = arr.reshape(3, 5) # 3×5 = 15، لكن لدينا 12 عنصر فقط
except ValueError as e:
    print(f"خطأ: {e}")
```

```
# مثال عملي: تحويل بيانات طالب
```

```
# لدينا 4 طلاب، كل طالب له 3 درجات
```

```
grades = np.array([85, 90, 78, 92, 88, 95, 78, 85, 82, 88, 92, 90])
print("\n=== مثال عملي: درجات الطلاب ===")
print("البيانات الأصلية (مسطحة):", grades)
```

```
# إعادة تشكيل: 4 طلاب × 3 مواد
```

```
grades_matrix = grades.reshape(4, 3)
print("\n(مصفوفة الدرجات) 4 طلاب × 3 مواد")
print(grades_matrix)
print(f"درجات الطالب الأول: {grades_matrix[0]}")
print(f"درجات المادة الأولى لجميع الطلاب: {grades_matrix[:, 0]}")
```


Flatten و Ravel - تسطيح المصفوفات

المبدأ والفكرة:

التسطيح Flattening هو عملية تحويل مصفوفة متعددة الأبعاد إلى مصفوفة أحادية البعد (1D). هناك طريقتان رئيسيتان:

flatten():

- تُنشئ نسخة جديدة من البيانات
- التعديل عليها لا يؤثر على المصفوفة الأصلية
- تستهلك ذاكرة إضافية

ravel():

- تُنشئ view إن أمكن (أسرع)
- التعديل عليها قد يؤثر على المصفوفة الأصلية
- أكثر كفاءة في الذاكرة

الاستخدامات:

- تحضير البيانات لخوارزميات ML
- تحويل الصور إلى متجهات
- عمليات حسابية بسيطة



Flatten و Ravel

تسطيح المصفوفات

```
import numpy as np
```

```
# إنشاء مصفوفة ثنائية البعد
```

```
matrix = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])
```

```
print("المصفوفة الأصلية:")
```

```
print(matrix)
```

```
print(f"الشكل: {matrix.shape}")
```

```
# ينشئ نسخة - flatten() استخدام 1
```

```
flattened = matrix.flatten()
```

```
print("\n=== باستخدام flatten() ===")
```

```
print("المصفوفة المسطحة:", flattened)
```

```
print(f"الشكل: {flattened.shape}")
```

```
# تعديل المصفوفة المسطحة
```

```
flattened[0] = 999
```

```
print("\nبعد تعديل flattened[0] = 999:")
```

```
print("المصفوفة المسطحة:", flattened)
```

```
print("المصفوفة الأصلية لم تتغير")
```

```
print(matrix) # لم تتأثر
```

```
# view ينشئ - ravel() استخدام 2
```

```
matrix2 = np.array([[10, 20, 30],
                   [40, 50, 60],
                   [70, 80, 90]])
```

```
raveled = matrix2.ravel()
```

```
print("\n=== باستخدام ravel() ===")
```

```
print("المصفوفة المسطحة:", raveled)
```

```
# تعديل المصفوفة المسطحة
```

```
raveled[0] = 999
```

```
print("\nraveled[0] = 999 بعد تعديل")
```

```
print("المصفوفة المسطحة:", raveled)
```

```
print("المصفوفة الأصلية تغيرت!")
```

```
print(matrix2) # تأثرت
```

نقل المصفوفات Transpose

المبدأ والفكرة:

النقل Transpose هو عملية قلب المصفوفة بحيث تصبح الصفوف أعمدة والأعمدة صفوف. في المصفوفات الرياضية، النقل يُرمز له بـ A^T .

الطرق المختلفة:

- T - اختصار سريع
- $\text{transpose}()$ - دالة كاملة

الاستخدامات:

- عمليات الجبر الخطي
- تحضير البيانات
- عمليات ضرب المصفوفي

```
import numpy as np
```

```
# مصفوفة بسيطة 2
```

```
matrix = np.array([[1, 2, 3],
                   [4, 5, 6]])
```

```
print("المصفوفة الأصلية (3x2):")
print(matrix)
print(f"الشكل: {matrix.shape}")
```

```
# 1. استخدام .T
```

```
transposed = matrix.T
print("\n=== باستخدام .T ===")
print("المصفوفة المنقولة (2x3):")
print(transposed)
print(f"الشكل: {transposed.shape}")
```

```
# 2. استخدام transpose()
```

```
transposed2 = matrix.transpose()
print("\n=== باستخدام transpose() ===")
print(transposed2)
```


مقدمة إلى البث Broadcasting

المبدأ والفكرة:

البث Broadcasting هو آلية قوية في NumPy تسمح بإجراء العمليات الحسابية بين مصفوفات ذات أشكال مختلفة دون الحاجة لنسخ البيانات يدوياً.

الفوائد:

- كود أنظف وأقصر
- أداء أفضل (لا حاجة للحلقات)
- استخدام أقل للذاكرة

قواعد البث:

1. إذا كانت المصفوفتان لهما نفس عدد الأبعاد، يتم المقارنة بُعد بُعد.
2. الأبعاد متوافقة إذا كانت:
 1. متساوية، أو
 2. أحدها يساوي 1

مقدمة إلى البث Broadcasting

```
import numpy as np
```

```
# مثال بسيط: إضافة قيمة ثابتة لمصفوفة
print("=== مثال 1: إضافة قيمة ثابتة ===")
arr = np.array([1, 2, 3, 4, 5])
print("المصفوفة:", arr)
```

```
# إضافة 10 لكل عنصر (البث التلقائي)
result = arr + 10
print("بعد إضافة 10:", result)
# NumPy [10, 10, 10, 10, 10] لتصبح [10, 10, 10, 10, 10]
```

```
# مثال 2: عمليات بين مصفوفة وقيمة
print("=== مثال 2: عمليات متعددة ===")
arr = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])
```

```
print("المصفوفة الأصلية:")
print(arr)
```

```
print("ضرب 2 ×:")
print(arr * 2)
```

```
print("قسمة 3 ÷:")
print(arr / 3)
```

```
print("أس 2:")
print(arr ** 2)
```

```
# مثال 3: إضافة صف إلى مصفوفة
print("=== مثال 3: إضافة صف ===")
matrix = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])
```

```
row = np.array([10, 20, 30])
```

```
print("المصفوفة 3×3:")
print(matrix)
print(f"الشكل: {matrix.shape}")
```

```
print("الصف 3:")
print(row)
print(f"الشكل: {row.shape}")
```

```
# الجمع - الصف يُبث لكل صف في المصفوفة
result = matrix + row
print("النتيجة:")
print(result)
print("الصف [10, 20, 30] أُضيف لكل صف في المصفوفة")
```

```
# مثال 4: إضافة عمود إلى مصفوفة
print("=== مثال 4: إضافة عمود ===")
matrix = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])
```

```
column = np.array([[10],
                  [20],
                  [30]])
```

```
print("المصفوفة 3×3:")
print(matrix)
```

```
print("العمود 1×3:")
print(column)
print(f"الشكل: {column.shape}")
```

مقدمة إلى الأقنعة المنطقية Boolean Masking

المبدأ والفكرة:

الأقنعة المنطقية Boolean Masking هي تقنية قوية لفلتر واختيار عناصر محددة من المصفوفة بناءً على شروط منطقية.

كيف تعمل:

1. نطبق شرط منطقي على المصفوفة
2. النتيجة: مصفوفة منطقية True/False
3. نستخدم هذه المصفوفة "كقناع"

لاختيار العناصر

الاستخدامات:

- فلتر البيانات
- البحث الشرطي
- تنظيف البيانات
- التحليل الإحصائي الشرطي

مقدمة إلى الأقنعة المنطقية Boolean Masking

```
import numpy as np
```

مصفوفة بسيطة

```
arr = np.array([10, 25, 30, 15, 40, 5, 35, 20])
```

```
print("=== المصفوفة الأصلية ===")
print(arr)
```

إنشاء قناع منطقي 1.

```
print("\n=== إنشاء قناع منطقي ===")
```

شرط: العناصر الأكبر من 20

```
mask = arr > 20
print(f"القناع (arr > 20):")
print(mask)
print(f"نوع البيانات: {mask.dtype}")
```

استخدام القناع لاختيار العناصر 2.

```
print("\n=== استخدام القناع ===")
filtered = arr[mask]
print(f"20 > العناصر: {filtered}")
```

أو مباشرة بدون متغير وسيط

```
print(f"20 < العناصر: {arr[arr < 20]}")
```

شروط منطقية متعددة 3.

```
print("\n=== شروط منطقية متعددة ===")
```

AND: &

العناصر بين 15 و 35

```
condition = (arr >= 15) & (arr <= 35)
print(f"35 و 15 بين العناصر: {arr[condition]}")
```

OR: |

العناصر أقل من 15 أو أكبر من 30

```
condition = (arr < 15) | (arr > 30)
print(f"30 > أو 15 < العناصر: {arr[condition]}")
```

NOT: ~

العناصر التي ليست بين 15 و 30

```
condition = ~(arr >= 15) & (arr <= 30)
print(f"30, 15 خارج نطاق: {arr[condition]}")
```

عدد العناصر المطابقة 4.

```
print("\n=== عدد العناصر ===")
count_greater_20 = (arr > 20).sum() # True = 1, False = 0
print(f"20 > عدد العناصر: {count_greater_20}")
```

```
count_between = ((arr >= 15) & (arr <= 35)).sum()
```

```
print(f"35 و 15 بين العناصر: {count_between}")
```

التحقق من الشروط 5.

```
print("\n=== التحقق من الشروط ===")
```

هل جميع العناصر > 0؟

```
all_positive = np.all(arr > 0)
print(f"{all_positive} هل جميع العناصر موجبة؟")
```

هل يوجد عنصر واحد على الأقل > 30؟

```
any_greater_30 = np.any(arr > 30)
print(f"{any_greater_30} هل يوجد عنصر > 30؟")
```

مصفوفات ثنائية البعد 6.

```
print("\n=== مصفوفات ثنائية البعد ===")
```

```
matrix = np.array([[10, 25, 30],
                   [15, 40, 5],
                   [35, 20, 12]])
```

```
print("المصفوفة:")
print(matrix)
```

قناع العناصر > 20

```
mask = matrix > 20
print("\n20 (> القناع:")
print(mask)
```

(D ستكون مصفوفة 1) اختيار العناصر

```
filtered = matrix[mask]
print(f"\n20 > العناصر: {filtered}")
```

مثال عملي: درجات الطلاب

```
print("\n=== مثال عملي: درجات الطلاب ===")
```

درجات 10 طلاب

```
grades = np.array([85, 92, 78, 65, 88, 45, 95, 72, 55, 90])
```

```
print(f"الدرجات: {grades}")
```

الطلاب الناجحون (≥ 60)

```
passing = grades[grades >= 60]
```

```
print(f"\n60(>= الناجحون: {passing})")
```

```
print(f"عدد الناجحين: {len(passing)}")
```

الطلاب الراسبون (< 60)

```
failing = grades[grades < 60]
```

```
print(f"\n60(< الراسبون: {failing})")
```

```
print(f"عدد الراسبين: {len(failing)}")
```

المتفوقون (≥ 90)

```
excellent = grades[grades >= 90]
```

```
print(f"\n90(>= المتفوقون: {excellent})")
```

الدرجات بين 70 و 85

```
middle_range = grades[(grades >= 70) & (grades < 85)]
```

```
print(f"\n85 و 70 الدرجات بين: {middle_range}")
```



تعديل العناصر باستخدام الأقنعة

المبدأ والفكرة:

الأقنعة لا تُستخدم فقط لاختيار العناصر، بل أيضاً لتعديلها بناءً على شروط.

```
import numpy as np
```

```
# مصفوفة درجات
```

```
grades = np.array([85, 45, 92, 35, 78, 65, 95, 58, 72, 50])
```

```
print("=== الدرجات الأصلية ===")
```

```
print(grades)
```

```
# تعديل العناصر المطابقة للشرط 1.
```

```
print("\n=== مثال 1: رفع الدرجات المنخفضة ===")
```

```
# رفع الدرجات الأقل من 60 إلى 60 (الدرجة الدنيا للنجاح)
```

```
grades_copy = grades.copy()
```

```
grades_copy[grades_copy < 60] = 60
```

```
print("بعد رفع الدرجات < 60 إلى 60")
```

```
print(grades_copy)
```

```
# إضافة قيمة للعناصر المطابقة 2.
```

```
print("\n=== مثال 2: درجات إضافية ===")
```

```
grades_copy = grades.copy()
```

```
# إضافة 5 درجات للطلاب الذين حصلوا على 90
```

```
grades_copy[grades_copy >= 90] += 5
```

```
# لكن لا نتجاوز 100
```

```
grades_copy[grades_copy > 100] = 100
```

```
print("بعد إضافة 5 درجات للمتفوقين")
```

```
print(grades_copy)
```

```
# 3. بديل أنيق - np.where() استخدام
```

```
print("\n=== np.where() مثال 3: استخدام ===")
```

```
# np.where(condition, value_if_true, value_if_false)
```

```
# إذا الدرجة >= 60 اكتبها كما هي، وإلا اكتبها 60
```

```
adjusted = np.where(grades >= 60, grades, 60)
```

```
print("np.where() باستخدام")
```

```
print(adjusted)
```


تعديل العناصر باستخدام الأقنعة (يتبع)

```
# مثال آخر: تصنيف النجاح/الرسوب
status = np.where(grades >= 60, "راسب", "راسب")
print("\nالحالة:")
print(status)

# 4. تعديلات متعددة الشروط
print("\n=== مثال 4: تعديلات متعددة ===")

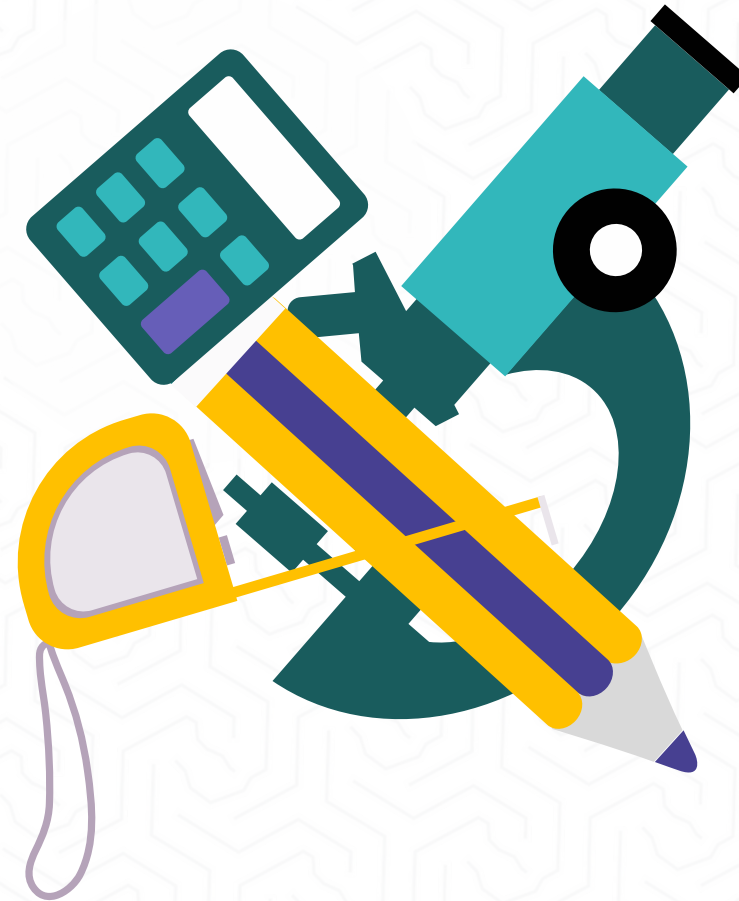
grades_copy = grades.copy()

# منح درجات إضافية حسب الفئة
# 90+: +5
# 80-89: +3
# 70-79: +2
# أقل من 70: +70

grades_copy[grades_copy >= 90] += 5
grades_copy[(grades_copy >= 80) & (grades_copy < 90)] += 3
grades_copy[(grades_copy >= 70) & (grades_copy < 80)] += 2

# التأكد من عدم تجاوز 100
grades_copy[grades_copy > 100] = 100

print("\n:بعد الدرجات الإضافية المتدرجة")
print(grades_copy)
```





Python for Everybody: Exploring Data in Python 3 Book by Charles Severance.

لکم شکراً





الكلية التطبيقية

الجامعة السعودية الإلكترونية

SAUDI ELECTRONIC UNIVERSITY

2011-1432

دبلوم الذكاء الاصطناعي التوليدي

اسم المقرر: البرمجة باستخدام بايثون

رمز المقرر: PYT103

الموضوع : الثامن



الموضوع الثامن: الدوال والكود المعياري

الجامعة الوطنية

الأهداف

في نهاية هذا الدرس سيكون الطالب قادرًا على أن:

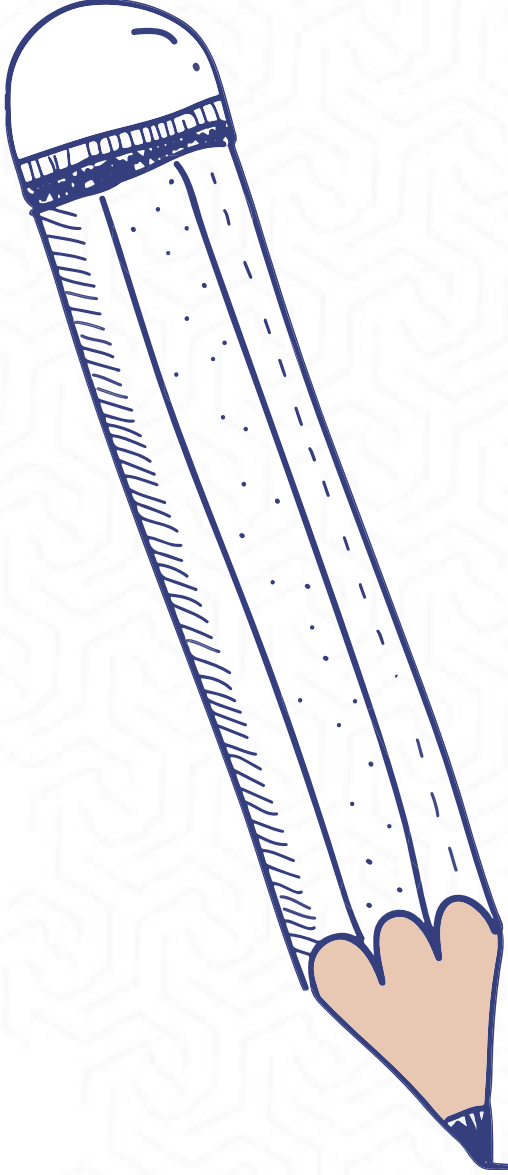
- ✓ يفهم مفهوم الدوال في بايثون وأهميتها.
- ✓ ينشئ الدوال ويستدعيها عند الحاجة.
- ✓ يميز بين الدوال المنتجة والدوال الخالية.
- ✓ يفسر معاملات الدوال ووسائطها.
- ✓ يطبق مبادئ إعادة استخدام الكود.
- ✓ يوظف التوابع الجاهزة في بايثون.
- ✓ يطبق مفاهيم التنقيح لحل المشكلات البرمجية.



وصف الدرس

في هذا الدرس سنتناول:

- تعلم الدوال في لغة بايثون كأحد أهم المفاهيم في البرمجة.
- كيفية إنشاء دوال مخصصة لحل مشاكل محددة
- كيفية استخدام الدوال الجاهزة في بايثون لتسريع عملية التطوير.
- مفاهيم المعاملات والوسائط
- كيفية إرجاع القيم من الدوال
- أفضل الممارسات في كتابة وتنظيم الكود لجعله قابلاً لإعادة الاستخدام والصيانة.



الموضوع الثامن | مقدمة في الدوال

ما هي الدوال؟

الدوال هي مجموعة منظمة من التعليمات البرمجية التي تؤدي مهمة محددة. تشبه الوصفات في الطبخ - مجموعة من الخطوات المرتبة لتحقيق نتيجة معينة.



لماذا نحتاج للدوال؟

تجنب التكرار:



كتابة الكود مرة واحدة واستخدامه عدة مرات

التنظيم:



تقسيم البرنامج إلى أجزاء صغيرة ومفهومة

سهولة الصيانة:

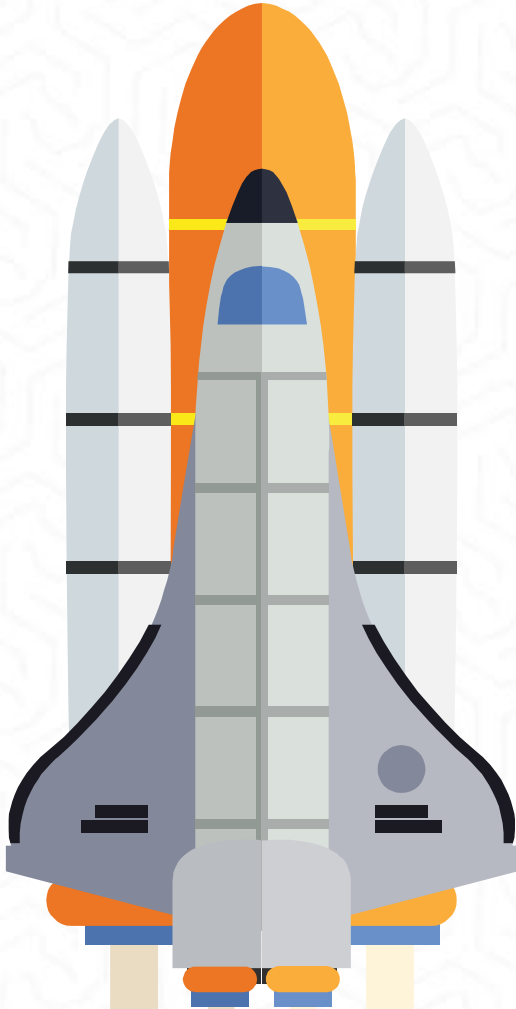


تعديل أو إصلاح جزء واحد بدلاً من البحث في كامل البرنامج

إعادة الاستخدام:



استخدام نفس الدالة في برامج مختلفة



تركيب الدالة في بايثون:

```
def function_name(parameters):  
    """
```

توثيق الدالة - وصف مختصر لما تفعله الدالة
 """

جسم الدالة - التعليمات البرمجية

return value # اختياري - إرجاع قيمة

مكونات الدالة:

الكلمة المفتاحية `def`: تعلن عن بداية تعريف الدالة

اسم الدالة: يجب أن يكون وصفيًا ومفهومًا

المعاملات: القيم التي تستقبلها الدالة (اختيارية)

النقطتان (:): تشير إلى بداية جسم الدالة

المسافة البادئة: جميع أسطر الدالة يجب أن تكون مزاحة بمسافة

إنشاء دالة بسيطة للترحيب:

```
def greet():  
    """
```

```
    دالة بسيطة لطباعة رسالة ترحيب  
    """
```

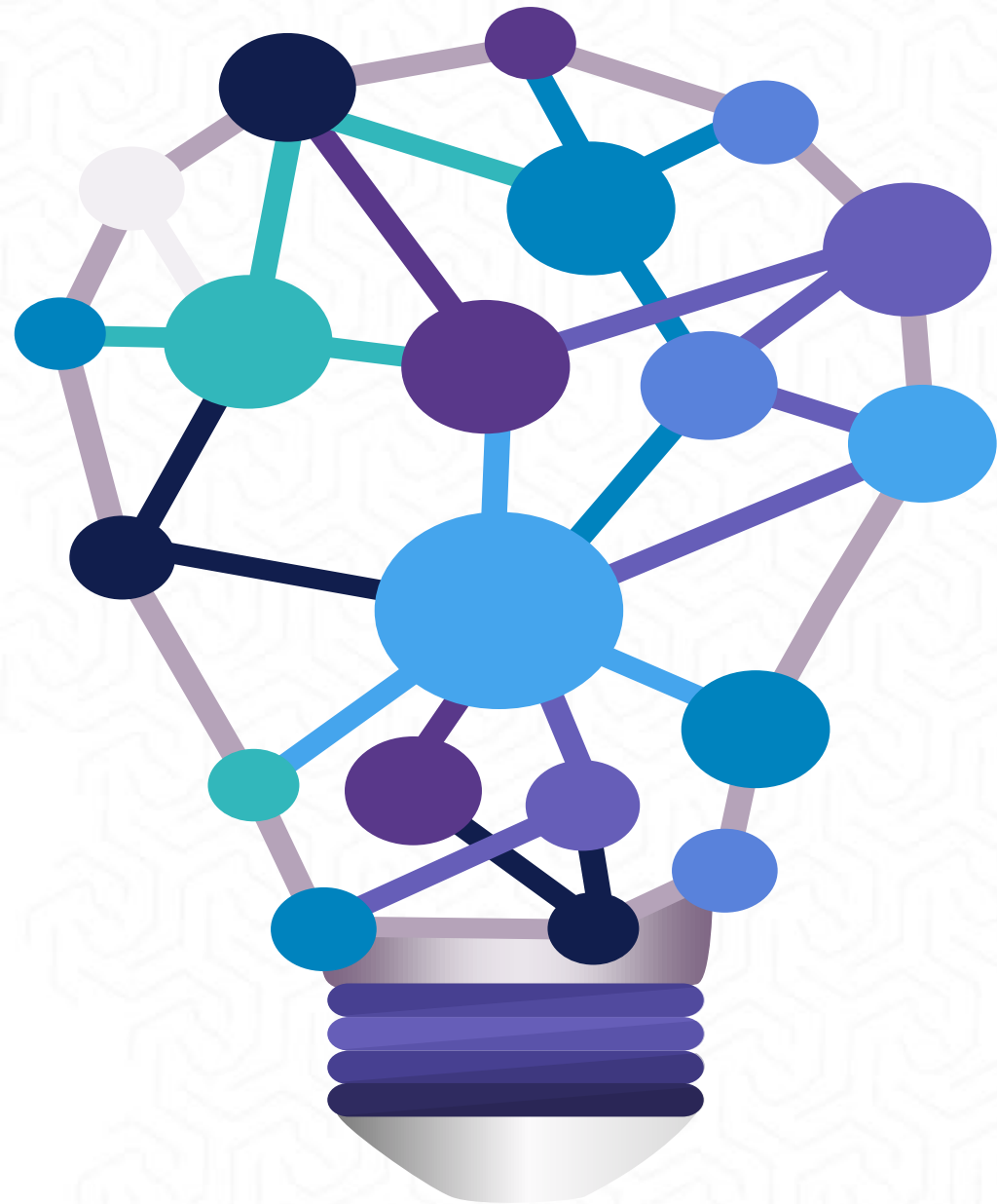
```
print("Welcome to Python Programming!") # طباعة رسالة ترحيب  
print("Let's learn functions together!") # طباعة رسالة تشجيعية
```

```
# استدعاء الدالة  
greet()
```

النتيجة:

Welcome to Python Programming!
Let's learn functions together!

ملاحظة: هذه دالة خالية (لا ترجع قيمة) وتنفذ مهمة الطباعة فقط.



الموضوع الثامن | الدوال مع المعاملات

المعاملات Parameters هي المتغيرات التي تستقبلها الدالة لتعمل معها:

```
def greet_person(name):  
    """  
    دالة تستقبل اسم شخص وترحب به  
    (نص) اسم الشخص: name  
    """  
    print(f"Hello, {name}!") # ترحيب شخصي  
    print(f"Nice to meet you, {name}") # رسالة لطيفة  
  
    # استدعاء الدالة مع وسائط مختلفة  
    greet_person("أحمد")  
    greet_person("فاطمة")  
    greet_person("محمد")
```

النتيجة:

Hello, أحمد!
Nice to meet you, أحمد
Hello, فاطمة!
Nice to meet you, فاطمة

الموضوع الثامن | دالة بمعاملات متعددة

يمكن للدالة أن تستقبل أكثر من معامل واحد:

```
def calculate_rectangle_area(length, width):  
    """  
    حساب مساحة المستطيل  
    length: طول المستطيل  
    width: عرض المستطيل  
    """  
    area = length * width # حساب المساحة  
    print(f"Rectangle dimensions: {length} x {width}") #  
    # طباعة الأبعاد  
    print(f"Area = {area} square units") # طباعة المساحة  
  
    # أمثلة على الاستخدام  
    calculate_rectangle_area(5, 3)  
    calculate_rectangle_area(10, 7)  
    calculate_rectangle_area(2.5, 4.2)
```

النتيجة:

- Rectangle dimensions: 5 x 3
- Area = 15 square units
- Rectangle dimensions: 10 x 7
- Area = 70 square units

الموضوع الثامن | الدوال التي ترجع قيماً استخدام return لإرجاع النتائج:

```
def add_numbers(a, b):  
    """  
    جمع رقمين وإرجاع النتيجة  
    الأرقام المراد جمعها: a, b  
    return: مجموع الرقمين  
    """  
    result = a + b # حساب المجموع  
    return result # إرجاع النتيجة  
  
def multiply_numbers(x, y):  
    """  
    ضرب رقمين وإرجاع النتيجة مباشرة  
    """  
    return x * y # حساب وإرجاع النتيجة مباشرة  
  
# استخدام الدوال وحفظ النتائج  
sum_result = add_numbers(15, 25)  
product_result = multiply_numbers(4, 7)  
  
print(f"15 + 25 = {sum_result}") # طباعة نتيجة الجمع  
print(f"4 × 7 = {product_result}") # طباعة نتيجة الضرب
```

النتيجة:

- $15 + 25 = 40$
- $4 \times 7 = 28$

الموضوع الثامن | القيم الافتراضية للمعاملات

يمكن إعطاء المعاملات قيماً افتراضية:

النتيجة:

```
def introduce_person(name, age=25, city="غير محدد"):
```

```
    """
    تعريف شخص مع قيم افتراضية للعمر والمدينة
    name: اسم الشخص (مطلوب)
    age: 25 (افتراضي) عمر الشخص
    city: "غير محدد" (افتراضي) مدينة الشخص
    """
```

```
    print(f"Name: {name}")          # طباعة الاسم
    print(f"Age: {age} years old")   # طباعة العمر
    print(f"City: {city}")          # طباعة المدينة
    print("-" * 20)                 # خط فاصل
```

```
    أمثلة مختلفة للاستدعاء #
```

```
    introduce_person("سارة")        # اسم فقط
    introduce_person("خالد", 30)      # اسم وعمر
    introduce_person("نور", 28, "الرياض") # جميع المعاملات
    introduce_person("علي", city="جدة") # اسم ومدينة فقط
```

- Name: سارة
- Age: 25 years old
- City: غير محدد
- -----
- Name: خالد
- Age: 30 years old
- City: غير محدد
- -----
- Name: نور
- Age: 28 years old
- City: الرياض
- -----
- Name: علي
- Age: 25 years old
- City: جدة
- -----

الموضوع الثامن | دالة حساب الدرجات

استخدام return لإرجاع النتائج:

```
def calculate_grade(score):  
    """  
    تحديد التقدير بناءً على الدرجة  
    score: 100 إلى 0 الدرجة من  
    return: التقدير (A, B, C, D, F)  
    """  
  
    if score >= 90:  
        grade = "A" # ممتاز  
    elif score >= 80:  
        grade = "B" # جيد جداً  
    elif score >= 70:  
        grade = "C" # جيد  
    elif score >= 60:  
        grade = "D" # مقبول  
    else:  
        grade = "F" # راسب  
  
    return grade
```

```
# اختبار الدالة مع درجات مختلفة  
student_scores = [95, 87, 74, 65, 45]  
student_names = ["خالد", "نور", "محمد", "فاطمة", "أحمد"]
```

```
for i in range(len(student_scores)):  
    score = student_scores[i] # الحصول على الدرجة  
    name = student_names[i] # الحصول على الاسم  
    grade = calculate_grade(score) # حساب التقدير  
    print(f"{name}: {score}% -> Grade {grade}") # طباعة النتيجة
```

مثال عملي - دالة تحديد التقدير بناءً على الدرجة:

- 95% -> Grade A : أحمد
- 87% -> Grade B : فاطمة
- 74% -> Grade C : محمد
- 65% -> Grade D : نور
- 45% -> Grade F : خالد

الموضوع الثامن | الدوال الرياضية

استخدام return لإرجاع النتائج:

التوابع الرياضية الجاهزة في بايثون:

استيراد مكتبة الرياضيات

```
def demonstrate_math_functions():
```

```
    """
```

عرض مختلف الدوال الرياضية الجاهزة

```
    """
```

```
    numbers = [16, 25, 9, 2.5, -7, 3.14159]
```

```
    print("Mathematical Functions Demo:")
```

```
    print("=" * 40)
```

```
    for num in numbers:
```

```
        print(f"\nNumber: {num}")
```

للأرقام الموجبة فقط) الجذر التربيعي

```
    if num > 0:
```

```
        sqrt_result = math.sqrt(num)
```

حساب الجذر التربيعي

```
        print(f" Square root: {sqrt_result:.2f}")
```

القيمة المطلقة

```
    abs_result = abs(num)
```

القيمة المطلقة

```
    print(f" Absolute value: {abs_result}")
```

التقريب

```
    rounded = round(num, 2)
```

التقريب لمنزلتين

```
    print(f" Rounded: {rounded}")
```

أكبر عدد صحيح أصغر من أو يساوي الرقم

```
    floor_result = math.floor(num)
```

```
    print(f" Floor: {floor_result}")
```

تشغيل العرض التوضيحي

```
demonstrate_math_functions()
```

Mathematical Functions Demo:

=====

Number: 16

Square root: 4.00

Absolute value: 16

Rounded: 16

Floor: 16

Number: 25

Square root: 5.00

Absolute value: 25

Rounded: 25

Floor: 25

Number: 9

Square root: 3.00

Absolute value: 9

Rounded: 9

Floor: 9

Number: 2.5

Square root: 1.58

Absolute value: 2.5

Rounded: 2.5

Floor: 2

Number: -7

Absolute value: 7

Rounded: -7

Floor: -7

Number: 3.14159

Square root: 1.77

Absolute value: 3.14159

Rounded: 3.14

Floor: 3

التوابع الجاهزة للتعامل مع النصوص:

```
def demonstrate_string_functions():  
    """  
    عرض دوال معالجة النصوص المختلفة  
    """  
    sample_text = " Hello Python Programming "  
  
    print("String Functions Demo:")  
    print("=" * 30)  
    print(f"Original: '{sample_text}'")  
  
    # إزالة المسافات الزائدة  
    # إزالة المسافات من البداية والنهاية  
    trimmed = sample_text.strip()  
    print(f"Trimmed: '{trimmed}'")  
  
    # تحويل إلى أحرف كبيرة  
    # تحويل إلى أحرف كبيرة  
    uppercase = trimmed.upper()  
    print(f"Uppercase: '{uppercase}'")  
  
    # تحويل إلى أحرف صغيرة  
    # تحويل إلى أحرف صغيرة  
    lowercase = trimmed.lower()  
    print(f"Lowercase: '{lowercase}'")  
  
    # تحويل أول حرف من كل كلمة إلى كبير  
    # تحويل إلى Title Case  
    title_case = trimmed.title()  
    print(f"Title Case: '{title_case}'")  
  
    # استبدال كلمة بأخرى  
    # استبدال كلمة  
    replaced = trimmed.replace("Python", "بايثون")  
    print(f"Replaced: '{replaced}'")  
  
    # تقسيم النص إلى كلمات  
    # تقسيم النص  
    words = trimmed.split()  
    print(f"Words: {words}")  
  
    # طول النص  
    # حساب طول النص  
    length = len(trimmed)  
    print(f"Length: {length} characters")  
  
    # تشغيل الدالة التوضيحية  
    demonstrate_string_functions()
```

String Functions Demo:

```
=====
Original: ' Hello Python Programming '
Trimmed: 'Hello Python Programming'
Uppercase: 'HELLO PYTHON PROGRAMMING'
Lowercase: 'hello python programming'
Title Case: 'Hello Python Programming'
Replaced: 'Hello بايثون Programming'
Words: ['Hello', 'Python', 'Programming']
Length: 24 characters
```


التوابع الجاهزة للتعامل مع القوائم:

```
def demonstrate_list_functions():  
    """  
    عرض دوال التعامل مع القوائم والمجموعات  
    """  
    numbers = [45, 12, 67, 23, 89, 34, 56]  
    fruits = ["apple", "banana", "orange", "grape"]  
  
    print("List Functions Demo:")  
    print("=" * 25)  
  
    # دوال إحصائية أساسية  
    print(f"Numbers: {numbers}")  
    print(f"Maximum: {max(numbers)}") # أكبر قيمة  
    print(f"Minimum: {min(numbers)}") # أصغر قيمة  
    print(f"Sum: {sum(numbers)}") # مجموع القيم  
    print(f"Length: {len(numbers)}") # عدد العناصر  
  
    # ترتيب القائمة  
    sorted_numbers = sorted(numbers) # ترتيب تصاعدي  
    print(f"Sorted: {sorted_numbers}")  
  
    sorted_desc = sorted(numbers, reverse=True) # ترتيب تنازلي  
    print(f"Sorted (desc): {sorted_desc}")  
  
    # دوال النصوص  
    print(f"Fruits: {fruits}")  
    longest_fruit = max(fruits, key=len) # أطول اسم فاكهة  
    print(f"Longest fruit name: {longest_fruit}")  
  
    # فلتر القائمة  
    large_numbers = list(filter(lambda x: x > 50, numbers)) # الأرقام أكبر من 50  
    print(f"Numbers > 50: {large_numbers}")  
  
    # تشغيل العرض التوضيحي  
    demonstrate_list_functions()
```

List Functions Demo:

```
=====  
Numbers: [45, 12, 67, 23, 89, 34, 56]  
Maximum: 89  
Minimum: 12  
Sum: 326  
Length: 7  
Sorted: [12, 23, 34, 45, 56, 67, 89]  
Sorted (desc): [89, 67, 56, 45, 34, 23, 12]  
  
Fruits: ['apple', 'banana', 'orange', 'grape']  
Longest fruit name: banana  
Numbers > 50: [67, 89, 56]
```

الموضوع الثامن | دالة البحث في القوائم

```
def find_student(students, target_name):  
    """  
    البحث عن طالب في قائمة الطلاب  
    قائمة أسماء الطلاب:  
    students:  
    target_name: اسم الطالب المراد البحث عنه  
    return: إذا لم يوجد 1- فهرس الطالب أو  
    """  
    for i in range(len(students)):  
        if students[i].lower() == target_name.lower(): # مقارنة غير حساسة لحالة الأحرف  
            return i # إرجاع الفهرس  
    return -1 # لم يوجد الطالب  
  
def search_and_display(student_list, search_name):  
    """  
    البحث عن طالب وعرض النتيجة  
    """  
    result = find_student(student_list, search_name) # البحث عن الطالب  
  
    if result != -1: # إذا وُجد الطالب  
        print(f"✓ Found '{search_name}' at position {result + 1}")  
    else: # إذا لم يوجد  
        print(f"X '{search_name}' not found in the list")  
  
    # قائمة الطلاب  
    students = ["سارة محمود", "نور حسن", "خالد أحمد", "فاطمة علي", "أحمد محمد"]  
  
    print("Student List:")  
    for i, student in enumerate(students, 1): # عرض قائمة الطلاب  
        print(f"{i}. {student}")  
  
    print("\nSearch Results:")  
    print("-" * 20)  
  
    # البحث عن طلاب مختلفين  
    search_names = ["يوسف كمال", "نور حسن", "محمد أحمد", "فاطمة علي"]  
  
    for name in search_names:  
        search_and_display(students, name) # البحث وعرض النتيجة
```

Student List:

1. أحمد محمد
2. فاطمة علي
3. خالد أحمد
4. نور حسن
5. سارة محمود

Search Results:

-
- ✓ Found 'فاطمة علي' at position 2
 - X 'محمد أحمد' not found in the list
 - ✓ Found 'نور حسن' at position 4
 - X 'يوسف كمال' not found in the list

استخدام try/except في الدوال للتعامل مع الأخطاء:

```
def safe_divide(a, b):  
    """  
    قسمة آمنة مع التعامل مع خطأ القسمة على صفر  
    المقسوم: a  
    المقسوم عليه: b  
    نتيجة القسمة أو رسالة خطأ  
    """  
    try:  
        # محاولة القسمة  
        result = a / b  
        return result  
    except ZeroDivisionError:  
        # التعامل مع القسمة على صفر  
        print("Error: Division by zero!")  
        return None  
    except TypeError:  
        # التعامل مع نوع بيانات خاطئ  
        print("Error: Invalid data type!")  
        return None  
  
def safe_square_root(number):  
    """  
    حساب الجذر التربيعي مع التعامل مع الأخطاء  
    """  
    try:  
        # التحقق من الرقم السالب  
        if number < 0:  
            raise ValueError("Cannot calculate square root of negative number")  
        import math  
        # حساب الجذر التربيعي  
        result = math.sqrt(number)  
        return result  
    except ValueError as e:  
        # التعامل مع القيم الخاطئة  
        print(f"Error: {e}")  
        return None  
    except TypeError:  
        # التعامل مع نوع البيانات الخاطئ  
        print("Error: Please provide a number!")  
        return None
```

```
# اختبار الدوال الآمنة  
print("Safe Division Tests:")  
print(f"10 ÷ 2 = {safe_divide(10, 2)}") # قسمة صحيحة  
print(f"10 ÷ 0 = {safe_divide(10, 0)}") # قسمة على صفر  
print(f"'10' ÷ 2 = {safe_divide('10', 2)}") # نوع بيانات خاطئ  
  
print("\nSquare Root Tests:")  
print(f"√25 = {safe_square_root(25)}") # جذر تربيعي صحيح  
print(f"√-4 = {safe_square_root(-4)}") # رقم سالب  
print(f"√'hello' = {safe_square_root('hello')}") # نوع بيانات خاطئ
```


الموضوع الثامن | تنظيم الكود باستخدام الدوال

مثال على تنظيم برنامج إدارة الطلاب:

```
def display_menu():
    """ عرض قائمة الخيارات الرئيسية """
    print("\n" + "=" * 30)
    print("Student Management System")
    print("=" * 30)
    print("1. Add student")
    print("2. Display all students")
    print("3. Search for student")
    print("4. Calculate class average")
    print("5. Exit")
    print("-" * 30)

def add_student(students, grades):
    """ إضافة طالب جديد """
    name = input("Enter student name: ") # إدخال اسم الطالب
    try:
        grade = float(input("Enter student grade: ")) # إدخال درجة الطالب
        students.append(name) # إضافة الاسم للقائمة
        grades.append(grade) # إضافة الدرجة للقائمة
        print(f"✓ Student '{name}' added successfully!")
    except ValueError:
        # التعامل مع خطأ إدخال الدرجة
        print("X Error: Please enter a valid grade!")

def display_students(students, grades):
    """ عرض جميع الطلاب ودرجاتهم """
    if not students:
        # التحقق من وجود طلاب
        print("No students in the system.")
        return

    print("\nAll Students:")
    print("-" * 25)
    for i in range(len(students)):
        print(f"{i+1}. {students[i]}: {grades[i]}")
```

```
def search_student(students, grades):
    """ البحث عن طالب معين """
    # التحقق من وجود طلاب
    print("No students in the system.")
    return

if not students:
    search_name = input("Enter student name to search: ")
    found = False # متغير لتتبع العثور على الطالب

    for i in range(len(students)):
        if students[i].lower() == search_name.lower(): # مقارنة غير حساسة للحالة
            print(f"✓ Found: {students[i]} - Grade: {grades[i]}")
            found = True
            break

    if not found:
        # إذا لم يوجد الطالب
        print(f"X Student '{search_name}' not found.")

def calculate_class_average(grades):
    """ حساب متوسط الصف """
    # التحقق من وجود درجات
    print("No grades available.")
    return

if not grades:
    average = sum(grades) / len(grades) # حساب المتوسط
    print(f"Class average: {average:.2f}")

# مثال على الاستخدام
students_list = []
grades_list = []

display_menu()
display_students(students_list, grades_list)
```

الموضوع الثامن | أفضل الممارسات في كتابة الدوال

```
# واضحة ومحددة المهمة -دالة جيدة ✓
def calculate_circle_area(radius):
    """
    حساب مساحة الدائرة

    Args:
        radius (float): نصف قطر الدائرة

    Returns:
        float: مساحة الدائرة

    Raises:
        ValueError: إذا كان نصف القطر سالب
    """
    if radius < 0:
        # التحقق من صحة المدخل
        raise ValueError("Radius cannot be negative")

    import math
    area = math.pi * radius ** 2
    return round(area, 2)

# دالة مع معاملات واضحة ✓
def format_student_info(name, age, grade, city="Unknown"):
    """
    تنسيق معلومات الطالب في نص منظم
    """
    info = f"""
    Student Information:
    -----
    Name: {name}
    Age: {age} years
    Grade: {grade}%
    City: {city}
    """
    return info.strip()

# إزالة المسافات الزائدة
```

```
# دالة تتعامل مع الأخطاء بشكل صحيح ✓
def safe_get_percentage(score, total):
    """
    حساب النسبة المئوية بطريقة آمنة
    """
    try:
        # التحقق من صحة المجموع
        if total <= 0:
            raise ValueError("Total must be positive")

        # حساب النسبة
        percentage = (score / total) * 100
        # ضمان النتيجة بين 0-100
        return min(100, max(0, percentage))

    except (TypeError, ZeroDivisionError) as e:
        # التعامل مع الأخطاء
        print(f"Error calculating percentage: {e}")
        return 0

# أمثلة على الاستخدام
print("Best Practices Examples:")
print("=" * 30)

# حساب مساحة الدائرة: مثال 1
try:
    area = calculate_circle_area(5)
    print(f"Circle area (radius=5): {area}")
except ValueError as e:
    print(f"Error: {e}")

# تنسيق معلومات طالب: مثال 2
student_info = format_student_info("أحمد محمد", 20, 85, "الرياض")
print(student_info)

# حساب النسبة المئوية: مثال 3
percentage = safe_get_percentage(85, 100)
print(f"Percentage: {percentage}%")

# حساب النسبة
```

إرشادات لكتابة دوال فعالة ومفهومة:

كيفية اختبار الدوال للتأكد من صحة عملها:

```
def test_math_functions():
    """
    اختبار دوال الحساب للتأكد من صحة عملها
    """
    print("Testing Math Functions:")
    print("=" * 25)

    # اختبار دالة الجمع
    def add(a, b):
        return a + b

    # حالات اختبار مختلفة
    test_cases = [
        (5, 3, 8), # أعداد موجبة
        (-2, 4, 2), # عدد سالب وموجب
        (0, 0, 0), # أصفار
        (-5, -3, -8) # أعداد سالبة
    ]

    print("Testing add function:")
    # متغير لتتبع نجاح الاختبارات
    all_passed = True

    for a, b, expected in test_cases:
        result = add(a, b) # تنفيذ الدالة
        passed = result == expected # مقارنة النتيجة المتوقعة
        status = "✓ PASS" if passed else "X FAIL" # تحديد حالة الاختبار

    print(f" add({a}, {b}) = {result} | Expected: {expected} | {status}")

    if not passed:
        # إذا فشل الاختبار
        all_passed = False

    print(f"\nOverall result: {'✓ All tests passed!' if all_passed else 'X Some tests failed!'}")

    def debug_function_example():
        """
        مثال على تنقيح دالة تحتوي على خطأ
        """
        def calculate_average_buggy(numbers):
            """
            دالة تحتوي على خطأ في حساب المتوسط
            """
            total = 0
            for num in numbers:
                total += num
            # بدلاً من طول القائمة + 1، القسمة على طول القائمة: خطأ
            # هنا الخطأ ←
            average = total / (len(numbers) + 1)
            return average
```

```
def calculate_average_fixed(numbers):
    """
    النسخة المصححة من الدالة
    """
    if not numbers:
        return 0

    # التحقق من القائمة الفارغة

    total = sum(numbers)
    average = total / len(numbers)
    return average

# حساب المجموع
# حساب المتوسط الصحيح

# اختبار النسختين
test_data = [10, 20, 30, 40, 50]
expected_avg = 30.0

print("\nDebugging Example:")
print("-" * 20)

buggy_result = calculate_average_buggy(test_data) # النسخة التي تحتوي على خطأ
fixed_result = calculate_average_fixed(test_data) # النسخة المصححة

print(f"Data: {test_data}")
print(f"Expected average: {expected_avg}")
print(f"Buggy result: {buggy_result} {'X' if buggy_result != expected_avg else '✓'}")
print(f"Fixed result: {fixed_result} {'X' if fixed_result != expected_avg else '✓'}")

# تشغيل الاختبارات
test_math_functions()
debug_function_example()
```


الموضوع الثامن | ملخص الدرس

✓ المفاهيم الأساسية:

تعريف الدوال وبنيتها الأساسية
الفرق بين الدوال المنتجة والخالية
معاملات الدوال والقيم الافتراضية

✓ التطبيقات العملية:

دوال الحسابات الرياضية
دوال معالجة النصوص والقوائم
دوال التحقق من صحة البيانات

✓ أفضل الممارسات:

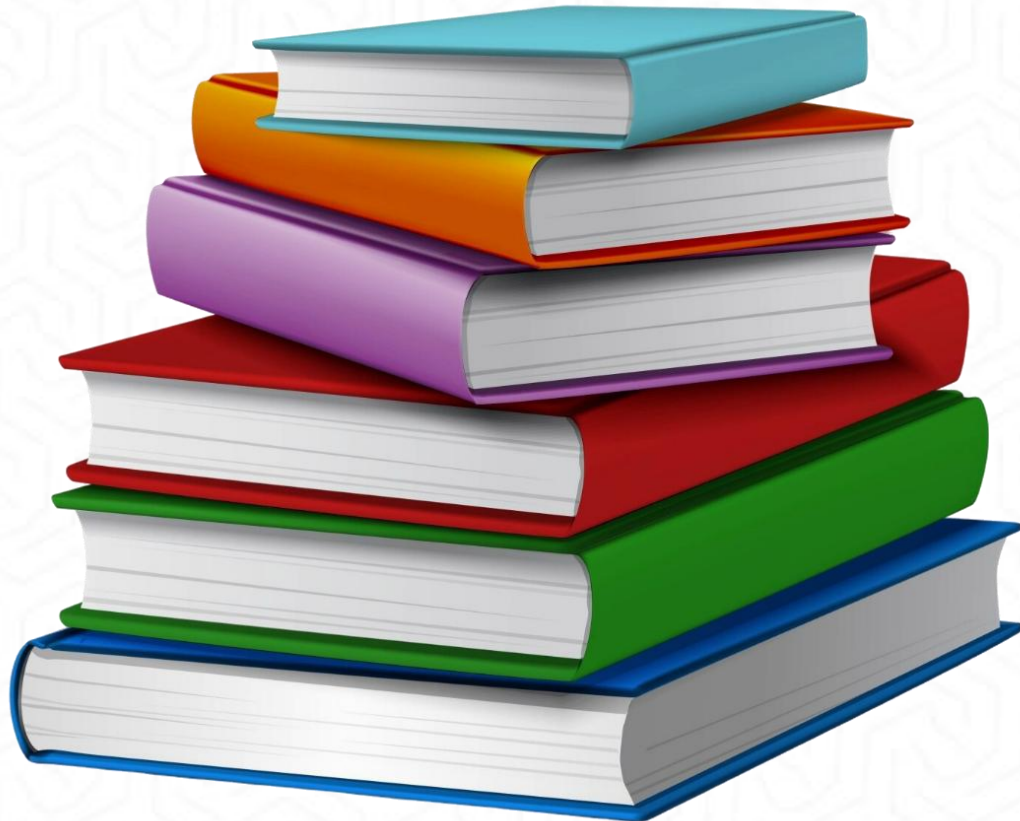
كتابة دوال واضحة ومحددة المهمة
التعامل مع الأخطاء باستخدام try/except
توثيق الدوال بشكل صحيح

✓ اختبار وتنقيح الدوال:

كتابة حالات اختبار شاملة
تقنيات تنقيح الأخطاء
مراجعة وتحسين الكود

الخطوات التالية:

التدرب على كتابة دوال أكثر تعقيداً
تعلم الوحدات والمكتبات في بايثون
دراسة البرمجة كائنية التوجه
بناء مشاريع تطبيقية شاملة



- Python for Everybody: Exploring Data in Python 3 Book by Charles Severance.



شكراً لكم



الكلية التطبيقية
الجامعة السعودية الإلكترونية
SAUDI ELECTRONIC UNIVERSITY
2011-1432

دبلوم الذكاء الاصطناعي التوليدي

اسم المقرر: البرمجة باستخدام بايثون

رمز المقرر: PYT103

الموضوع: الحادي عشر

مقدمة إلى NumPy المصفوفات والعمليات

11

الموضوع الحادي عشر



الأهداف

في نهاية هذا الدرس سيكون الطالب قادرًا على أن:

- يفهم ماهية مكتبة NumPy وأهميتها في الحوسبة العلمية وتحليل البيانات.
- يُنشئ المصفوفات (Arrays) بمختلف أنواعها وأشكالها.
- يتعامل مع خصائص المصفوفات وأبعادها باستخدام أدوات NumPy.
- يطبّق العمليات الحسابية الأساسية على المصفوفات بكفاءة.
- يستخدم دوال NumPy في إجراء التحليلات الإحصائية.
- يُنفّذ أساليب الفهرسة والتقطيع لمعالجة البيانات داخل المصفوفات.



وصف الدرس

في هذا الدرس سنتناول:

- أهمية مكتبة NumPy كأداة أساسية في الحوسبة العلمية بلغة Python.
- التعرف على بنية البيانات ndarray واستخدامها في معالجة البيانات متعددة الأبعاد.
- إنشاء المصفوفات البسيطة والمتعددة الأبعاد واستخدامها في التطبيقات العملية.
- تنفيذ العمليات الحسابية الأساسية بكفاءة وسرعة عالية.
- تطبيق تقنيات الفهرسة والتقطيع (Indexing & Slicing) لمعالجة البيانات.
- استخدام الدوال الإحصائية المدمجة في NumPy لتحليل البيانات.

ما هي NumPy؟

المبدأ والفكرة

NumPy هي مكتبة Python أساسية للحوسبة العلمية. تم تطويرها لحل مشكلة بطء العمليات الحسابية على القوائم العادية في Python. توفر NumPy نوع بيانات خاص يسمى `ndarray` (n-dimensional array) وهو مصفوفة متعددة الأبعاد تخزن عناصر من نفس النوع.

هذه المصفوفات
أسرع بكثير من القوائم
العادية لأنها:

- مخزنة في ذاكرة متجاورة contiguous memory.
- مكتوبة بلغة C في الخلفية.
- تدعم العمليات الشعاعية vectorized operations .
- تستهلك ذاكرة أقل.

المفاهيم الأساسية

المصطلحات الرئيسية:

1. **الصف Class**: قالب أو مخطط لإنشاء الكائنات.
2. **الكائن Object**: نسخة محددة من الصف
3. **الخصائص Attributes**: البيانات المخزنة في الكائن
4. **الدوال Methods**: الوظائف المرتبطة بالكائن

مثال من الحياة

- **الصف**: مخطط السيارة
- **الكائن**: سيارتك الشخصية
- **الخصائص**: اللون، الموديل، السرعة
- **الدوال**: تشغيل، إيقاف، تسريع

الفرق بين القائمة العادية والمصفوفة

تثبيت NumPy واستيراد

```
import numpy as np
```

إنشاء قائمة عادية

```
python_list = [1, 2, 3, 4, 5]
```

```
print("القائمة العادية:", python_list)
```

```
print("نوع القائمة:", type(python_list))
```

إنشاء مصفوفة NumPy

```
numpy_array = np.array([1, 2, 3, 4, 5])
```

```
print("\nNumPy مصفوفة:", numpy_array)
```

```
print("نوع المصفوفة:", type(numpy_array))
```

المقارنة في الحجم

```
import sys
```

```
print("\nحجم القائمة:", sys.getsizeof(python_list), "bytes")
```

```
print("حجم المصفوفة:", numpy_array.nbytes, "bytes")
```

إنشاء المصفوفات الأساسية

المبدأ والفكرة

هناك عدة طرق لإنشاء مصفوفات NumPy.

الطريقة الأكثر شيوعاً هي استخدام الدالة `np.array()` التي تحول قائمة Python إلى مصفوفة NumPy.

يمكن إنشاء مصفوفات أحادية البعد D1، ثنائية البعد D2، أو متعددة الأبعاد nD).

عند إنشاء المصفوفة، يقوم NumPy تلقائياً بتحديد نوع البيانات الأنسب، لكن يمكنك تحديد النوع يدوياً باستخدام المعامل `dtype`.

```
import numpy as np
```

```
# مصفوفة أحادية البعد
```

```
arr_1d = np.array([10, 20, 30, 40, 50])  
print("مصفوفة 1D:", arr_1d)
```

```
# (مصفوفة 2x3) مصفوفة ثنائية البعد
```

```
arr_2d = np.array([[1, 2, 3],  
                  [4, 5, 6]])  
print("\nمصفوفة 2D:\n", arr_2d)
```

```
# مصفوفة ثلاثية البعد
```

```
arr_3d = np.array([[[1, 2], [3, 4]],  
                  [[5, 6], [7, 8]]])  
print("\nمصفوفة 3D:\n", arr_3d)
```

```
# تحديد نوع البيانات
```

```
arr_float = np.array([1, 2, 3], dtype=float)  
print("\nمصفوفة أعداد عشرية:", arr_float)
```


دوال إنشاء المصفوفات الخاصة

المبدأ والفكرة

```
import numpy as np
```

```
# مصفوفة من الأصفار (3x4)
zeros = np.zeros((3, 4))
print("\nمصفوفة الأصفار", zeros)
```

```
# مصفوفة من الآحاد (2x3)
ones = np.ones((2, 3))
print("\nمصفوفة الآحاد", ones)
```

```
# مصفوفة مملوءة بقيمة محددة
sevens = np.full((2, 2), 7)
print("\nمصفوفة السبعات", sevens)
```

```
# مصفوفة الوحدة (Identity Matrix)
identity = np.eye(3)
print("\nمصفوفة الوحدة", identity)
```

توفر NumPy دوال مساعدة لإنشاء مصفوفات بقيم محددة مسبقاً. هذه الدوال مفيدة جداً عند الحاجة لتهيئة مصفوفات بأحجام معينة:

- **zeros()**: تنشئ مصفوفة مملوءة بالأصفار
- **ones()**: تنشئ مصفوفة مملوءة بالآحاد
- **empty()**: تنشئ مصفوفة غير مهياً (قيم عشوائية)
- **full()**: تنشئ مصفوفة مملوءة بقيمة محددة
- **eye()** أو **identity()**: تنشئ مصفوفة الوحدة (Identity Matrix)

إنشاء مصفوفات بمدى محدد

المبدأ والفكرة

عند الحاجة لإنشاء تسلسلات رقمية، توفر NumPy دوال متخصصة:

- **arange()**: مشابهة لدالة **range()** في Python ، تنشئ قيم من بداية إلى نهاية بخطوة معينة
- **linspace()**: تنشئ عدد محدد من القيم موزعة بالتساوي بين قيمتين
- **logspace()**: تنشئ قيم موزعة بشكل لوغاريتمي

الفرق الأساسي بين **arange()** و **linspace()** هو أن الأولي تحدد الخطوة بينما الثانية تحدد عدد العناصر.

```
import numpy as np
```

```
# (2 بخطوة 10 إلى 0 من) باستخدام arange
arr_range = np.arange(0, 10, 2)
print("arange:", arr_range)
```

```
# مع أعداد عشرية باستخدام arange
arr_range_float = np.arange(0, 1, 0.2)
print("arange عشرية:", arr_range_float)
```

```
# (5 إلى 0 قيم من 10) باستخدام linspace
arr_linspace = np.linspace(0, 5, 10)
print("\nlinspace:", arr_linspace)
```

```
# (10^0 إلى 10^2 قيم من 5) باستخدام logspace
arr_logspace = np.logspace(0, 2, 5)
print("\nlogspace:", arr_logspace)
```

المصفوفات العشوائية

المبدأ والفكرة

```
import numpy as np
```

```
# للحصول على نتائج قابلة للتكرار seedتحديد  
np.random.seed(42)
```

```
# 0 و 1 قيم عشوائية بين  
random_arr = np.random.random((2, 3))  
print("\n0 و 1 قيم عشوائية بين", random_arr)
```

```
# 1 إلى 10 أعداد صحيحة عشوائية من  
random_int = np.random.randint(1, 11, size=(2, 4))  
print("\nأعداد صحيحة عشوائية", random_int)
```

```
# قيم من التوزيع الطبيعي  
normal_dist = np.random.randn(3, 3)  
print("\nتوزيع طبيعي", normal_dist)
```

```
# اختيار عشوائي  
choices = np.random.choice(['A', 'B', 'C'], size=5)  
print("\nاختيارات عشوائية:", choices)
```

توفر وحدة random في NumPy مجموعة من الدوال لإنشاء مصفوفات بقيم عشوائية.

هذه الدوال مفيدة جداً في المحاكاة والاختبار والتعلم الآلي:

random(): قيم عشوائية بين 0 و 1

randint(): أعداد صحيحة عشوائية في مدى محدد

randn(): قيم من التوزيع الطبيعي المعياري

choice(): اختيار عشوائي من مصفوفة موجودة

يمكن تحديد seed لضمان إعادة إنتاج نفس النتائج العشوائية.

خصائص المصفوفات

المبدأ والفكرة

```
import numpy as np
```

```
# إنشاء مصفوفة ثنائية البعد  
arr = np.array([[1, 2, 3, 4],  
               [5, 6, 7, 8],  
               [9, 10, 11, 12]])
```

```
print("المصفوفة:\n", arr)  
print("\nالخصائص:")  
print("الشكل(shape):", arr.shape) # (3, 4) - 4 صفوف وأعمدة 3  
print("عدد الأبعاد(ndim):", arr.ndim) # 2 - بُعدين  
print("عدد العناصر(size):", arr.size) # 12 عنصر  
print("نوع البيانات(dtype):", arr.dtype) # int64 أو int32  
print("حجم العنصر(itemsizes):", arr.itemsize, "bytes")  
print("الذاكرة الكلية(nbytes):", arr.nbytes, "bytes")
```

كل مصفوفة NumPy لها مجموعة من الخصائص المهمة التي تصف هيكلها:

- **shape:** شكل المصفوفة (عدد العناصر في كل بُعد)
- **ndim:** عدد الأبعاد
- **size:** إجمالي عدد العناصر
- **dtype:** نوع البيانات المخزنة
- **itemsize:** حجم كل عنصر بالبايتات
- **nbytes:** إجمالي الذاكرة المستهلكة

فهم هذه الخصائص ضروري للتعامل الصحيح مع البيانات وتحسين الأداء.

الفهرسة والوصول للعناصر

المبدأ والفكرة

الفهرسة في NumPy مشابهة للفهرسة في القوائم العادية، لكن مع إمكانيات موسعة للمصفوفات متعددة الأبعاد. يمكن:

- الوصول لعنصر واحد باستخدام الفهارس
 - الوصول لصف أو عمود كامل
 - استخدام الفهرسة السالبة للبدء من النهاية
 - للمصفوفات متعددة الأبعاد، نستخدم فهرس لكل بُعد مفصول بفاصلة
- الفهرسة تبدأ من 0 كما في Python.

```
import numpy as np
```

مصفوفة أحادية البعد

```
arr_1d = np.array([10, 20, 30, 40, 50])
print("1D المصفوفة:", arr_1d)
print("العنصر الأول:", arr_1d[0])
print("العنصر الأخير:", arr_1d[-1])
```

مصفوفة ثنائية البعد

```
arr_2d = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])
print("\n2D المصفوفة:\n", arr_2d)
print("العمود 0 العنصر في الصف 1:", arr_2d[0, 1]) # 2
print("العمود 2 العنصر في الصف 2:", arr_2d[2, 2]) # 9
print("الصف الأول كامل:", arr_2d[0])             # [1, 2, 3]
print("العمود الثاني:", arr_2d[:, 1])             # [2, 5, 8]
```

التقطيع Slicing

المبدأ والفكرة

التقطيع يسمح باستخراج أجزاء من المصفوفة. الصيغة العامة حيث: start:stop:step هي

- start: الفهرس الابتدائي (افتراضياً 0)
- stop: الفهرس النهائي (غير مشمول)
- step: الخطوة (افتراضياً 1)

يمكن تطبيق التقطيع على كل بُعد بشكل منفصل.

ملاحظة مهمة: التقطيع ينشئ "view" وليس نسخة، لذا تعديل القطعة يعدل المصفوفة الأصلية.

```
import numpy as np
```

مصفوفة أحادية البعد

```
arr = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
print("المصفوفة الأصلية:", arr)
print("5: إلى 2 العناصر من", arr[2:6]) # [2, 3, 4, 5]
print("5: العناصر من البداية إلى", arr[:5]) # [0, 1, 2, 3, 4]
print("5: إلى النهاية 5 العناصر من", arr[5:]) # [5, 6, 7, 8, 9]
print("كل عنصر ثاني", arr[::2]) # [0, 2, 4, 6, 8]
```

مصفوفة ثنائية البعد

```
arr_2d = np.array([[1, 2, 3, 4],
                   [5, 6, 7, 8],
                   [9, 10, 11, 12]])
print("\nالمصفوفة 2D:\n", arr_2d)
print("\nأول صفين، أول عمودين", arr_2d[:2, :2])
print("\n2-3: الأعمدة 1-2 الصفوف", arr_2d[1:3, 2:4])
```

العمليات الحسابية الأساسية

المبدأ والفكرة

NumPy يدعم العمليات الحسابية العنصرية (element-wise operations) حيث تُطبق العملية على كل عنصر مقابل. هذه العمليات محسّنة وأسرع بكثير من استخدام الحلقات التكرارية.

تشمل العمليات:

- الجمع (+)، الطرح (-)، الضرب (*)، القسمة (/)
- الأس (**)، الباقي (%)، القسمة الصحيحة (//)
- العمليات مع قيم ثابتة)
- العمليات بين مصفوفتين بنفس الشكل

```
import numpy as np
```

```
arr1 = np.array([10, 20, 30, 40])  
arr2 = np.array([1, 2, 3, 4])
```

```
print("1:المصفوفة", arr1)  
print("2:المصفوفة", arr2)
```

عمليات بين مصفوفتين

```
print("\nالجمع:", arr1 + arr2)      # [11, 22, 33, 44]  
print("الطرح:", arr1 - arr2)      # [9, 18, 27, 36]  
print("الضرب:", arr1 * arr2)      # [10, 40, 90, 160]  
print("القسمة:", arr1 / arr2)      # [10., 10., 10., 10.]
```

عمليات مع قيمة ثابتة

```
print("\n2:ضرب بـ", arr1 * 2)      # [20, 40, 60, 80]  
print("5:إضافة", arr1 + 5)      # [15, 25, 35, 45]  
print("2:القوة", arr2 ** 2)        # [1, 4, 9, 16]
```

الدوال الإحصائية

المبدأ والفكرة

NumPy يوفر مجموعة شاملة من الدوال الإحصائية التي يمكن تطبيقها على المصفوفة كاملة أو على محور معين (axis).
المحور 0 يمثل الصفوف والمحور 1 يمثل الأعمدة.

الدوال الإحصائية الأساسية:

- `mean()`: المتوسط الحسابي
- `median()`: الوسيط
- `std()`: الانحراف المعياري
- `var()`: التباين
- `min()`, `max()`: القيم الدنيا والعليا
- `sum()`, `prod()`: المجموع والضرب

```
import numpy as np
```

```
# مصفوفة درجات الطلاب (4 طلاب، 3 مواد)
```

```
grades = np.array([[85, 90, 78],  
                  [92, 88, 95],  
                  [78, 85, 82],  
                  [88, 92, 90]])
```

```
print("n درجات الطلاب:", grades)
```

```
# إحصائيات على المصفوفة كاملة
```

```
print("\n", "np.mean(grades):", np.mean(grades))  
print(" ", "np.max(grades):", np.max(grades))  
print(" ", "np.min(grades):", np.min(grades))  
print(" ", "np.std(grades):", np.std(grades))
```

```
# (الأعمدة) - axis=1 إحصائيات لكل طالب
```

```
print("\n", "np.mean(grades, axis=1):", np.mean(grades, axis=1))
```

```
# (الصفوف) - axis=0 إحصائيات لكل مادة
```

```
print(" ", "np.mean(grades, axis=0):", np.mean(grades, axis=0))
```


حفظ وتحميل البيانات

المبدأ والفكرة

NumPy يوفر طرق فعالة لحفظ وتحميل المصفوفات من وإلى الملفات:

للملفات الثنائية (أسرع وأكفاً):

- `save()`: حفظ مصفوفة واحدة بصيغة `.npy`.
- `load()`: تحميل مصفوفة من ملف `.npy`.
- `savez()`: حفظ عدة مصفوفات في ملف مضغوط `.npz`.

للملفات النصية:

- `savetxt()`: حفظ كملف نصي
- `loadtxt()`: تحميل من ملف نصي
- مفيد للتوافق مع برامج أخرى

```
import numpy as np

# إنشاء بيانات للحفظ
data = np.array([[1, 2, 3],
                 [4, 5, 6],
                 [7, 8, 9]])

print("البيانات الأصلية:\n", data)

# حفظ بصيغة ثنائية
np.save('my_data.npy', data)
print("\nملف my_data.npy تم حفظ الملف")

# تحميل من الملف الثنائي
loaded_data = np.load('my_data.npy')
print("البيانات المحملة:\n", loaded_data)

# حفظ كملف نصي CSV
np.savetxt('my_data.csv', data, delimiter=',', fmt='%d')
print("\nملف my_data.csv تم حفظ الملف")

# تحميل من ملف CSV
loaded_csv = np.loadtxt('my_data.csv', delimiter=',')
print("البيانات CSV:\n", loaded_csv)

# حفظ عدة مصفوفات
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
np.savez('multiple.npz', first=arr1, second=arr2)
print("\nملف multiple.npz تم حفظ ملف متعدد")
```

المراجع



- ❑ Python for Everybody: Exploring Data in Python 3
Book by Charles Severance.